G L Bajaj Group of Institutions, Mathura

Worksheet Problems on Data Structures and Algorithms

Worksheet 10

Q1 Detect Loop in linked list

Given the head of a singly linked list, the task is to check if the linked list has a loop. A loop means that the last node of the linked list is connected back to a node in the same list. So if the next of the last node is null. then there is no loop.

Note: You need to return a boolean value true if there is a loop, otherwise false.

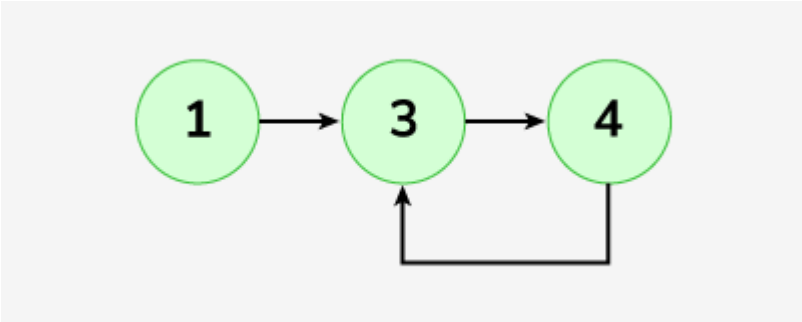
The following is an internal representation of every test case (three inputs).
A LinkedList and a pos (1-based index)-Position of the node to which the last node links back if there is a loop. If the linked list does not have any loop, then pos = 0.

Examples:

Input: LinkedList: 1->3->4

Output: true

Explanation:

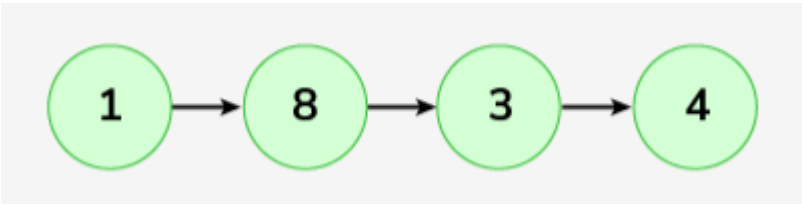


See the above list there exists a loop connecting the last node back to the second node.

Input: LinkedList: 1->8->3->4

Output: false

Explanation:

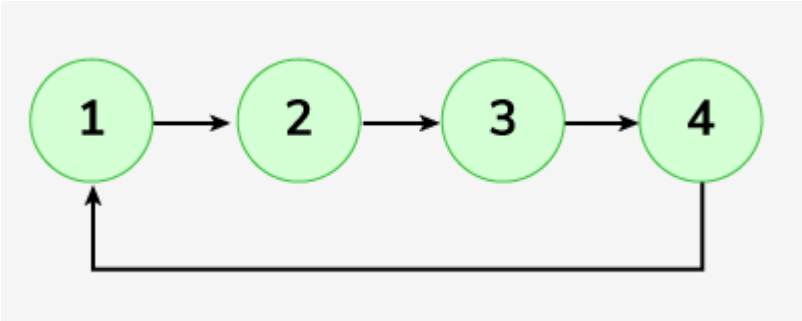


There is no loop exists.

Input: LinkedList: 1->2->3->4

Output: true

Explanation:



A loop is present connecting the last node back to the first node.

Q2 Remove loop in Linked List

Given the head of a linked list that may contain a loop. A loop means that the last node of the linked list is connected back to a node in the same list. So if the next of the previous node is null. then there is no loop. Remove the loop from the linked list, if it is present (we mainly need to make the next of the last node null). Otherwise, keep the linked list as it is.

Note: Given an integer, pos (1 based index) Position of the node to which the last node links back if there is a loop. If the linked list does not have any loop, then pos = 0.

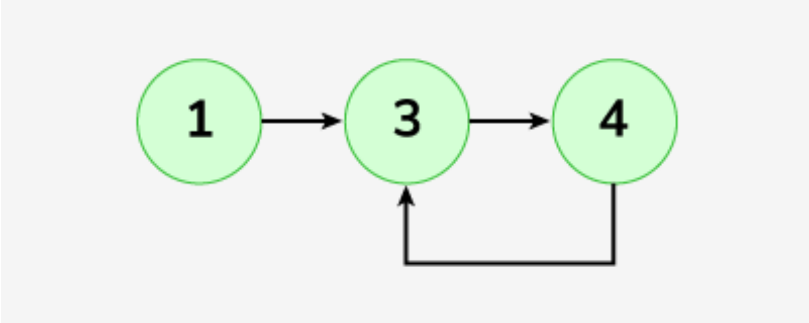
The generated output will be true if your submitted code is correct, otherwise, false.

Examples:

Input: Linked list: 1->3->4, pos = 2

Output: true

Explanation: The linked list looks like

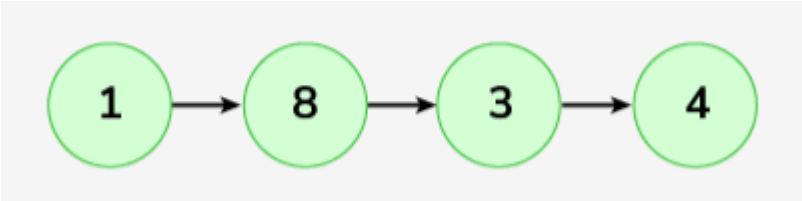


A loop is present. If you remove it successfully, the answer will be true.

Input: Linked list: 1->8->3->4, pos = 0

Output: true

Explanation:

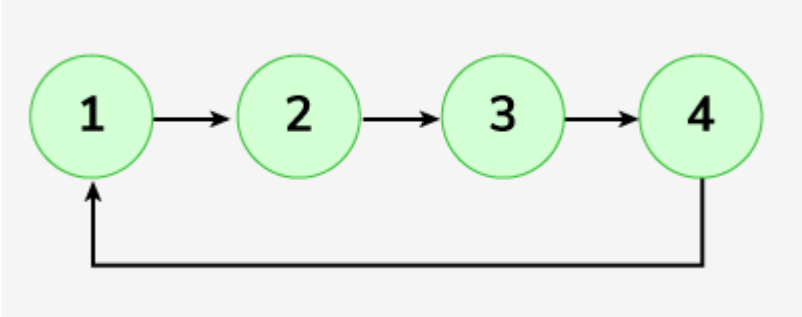


The Linked list does not contains any loop.

Input: Linked list: 1->2->3->4, pos = 1

Output: true

Explanation: The linked list looks like



Q3 Kth from End of Linked List

Given the head of a linked list and the number k, Your task is to find the kth node from the end. If k is more than the number of nodes, then the output should be -1.

Examples

Input: LinkedList: 1->2->3->4->5->6->7->8->9, k = 2

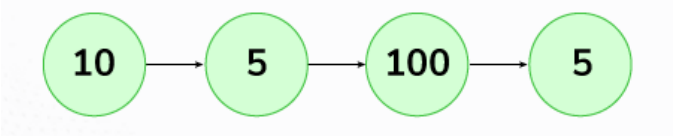
Output: 8

Explanation: The given linked list is 1->2->3->4->5->6->7->8->9. The 2nd node from end is 8.

Input: LinkedList: 10->5->100->5, k = 5

Output: -1

Explanation: The given linked list is 10->5->100->5. Since 'k' is more than the number of nodes, the output is -1.

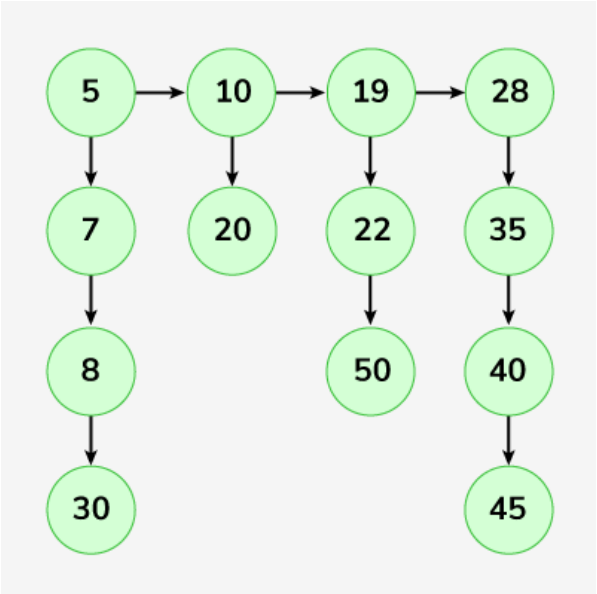


Given a Linked List, where every node represents a sub-linked-list and contains two pointers:
(i) a next pointer to the next node,
(ii) a bottom pointer to a linked list where this node is head.
Each of the sub-linked lists is in sorted order.
Flatten the Link List so all the nodes appear in a single level while maintaining the sorted order.

Note: The flattened list will be printed using the bottom pointer instead of the next pointer. Look at the printList() function in the driver code for more clarity.

Examples:

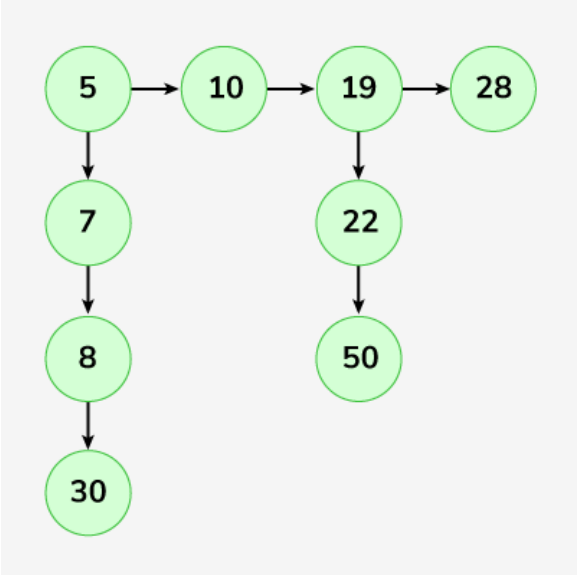
Input:



Output: 5-> 7-> 8- > 10 -> 19-> 20-> 22-> 28-> 30-> 35-> 40-> 45-> 50.

Explanation: The resultant linked lists has every node in a single level.(Note: | represents the bottom pointer.)

Input:



Output: 5-> 7-> 8-> 10-> 19-> 22-> 28-> 30-> 50

Explanation: The resultant linked lists has every node in a single level.(Note: | represents the bottom pointer.)

Q5 Merge two sorted linked lists

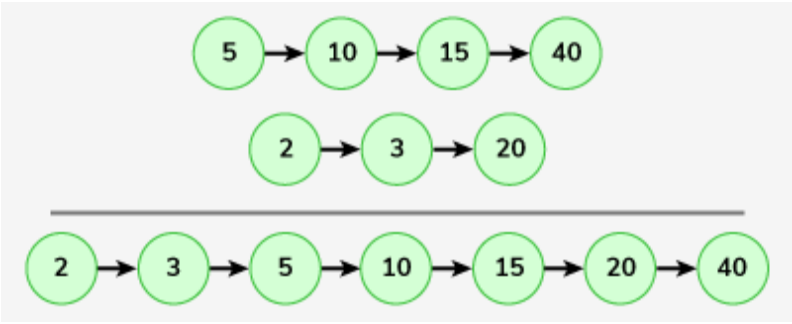
Given two sorted linked lists consisting of nodes respectively. The task is to merge both lists and return the head of the merged list.

Examples:

Input: LinkedList1: 5->10->15->40, ListedList2: 2->3->20

Output: 2->3->5->10->15->20->40

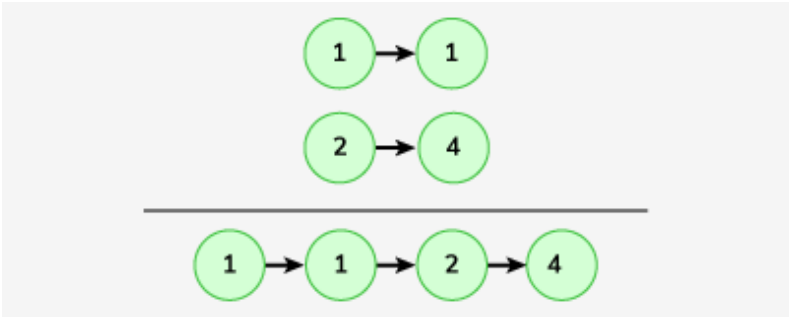
Explanation:



Input: LinkedList1: 1->1, LinkedList2: 2->4

Output: 1->1->2->4

Explanation:


1
1
2
4
1
1
2
4